

The five questions, explained plainly

Each one asks whether a single missing ability is what random play lacks. They do not depend on each other, so nobody waits for anybody. One person, one question.

First, the thing they all share

Our agent today plays **randomly**. It picks a button or a screen position at random, presses it, looks at the screen, and remembers moves that appeared to do nothing so it can avoid repeating them. On 25 games with 4 attempts each, it clears about **8 levels out of 100 attempts**.

Every question below has the same shape: *add one ability, keep everything else identical, and see whether it clears more levels than the version without it*. The version without it is called the **control**. Without a control you cannot tell whether your idea helped or the games were just easier that day.

Five capabilities have already been added to this agent. None of them beat random play.

That is not a discouraging fact, it is the reason this work is worth doing. If a plain statistical trick on a laptop beats an expensive reasoning system, then the expensive system was never necessary. These questions are the cheap controls that make that claim mean something.

Do we misjudge when the screen changed, because counters tick?

THE WORRY

The agent decides "did my move do anything?" by comparing a short code made from the screen's pixels, before and after. If the code changes, it says the move did something.

But imagine the game has a move-counter or a timer on screen that ticks on *every* move. Then the screen *always* looks different, so every move looks useful, and the whole "skip the useless moves" trick quietly stops working.

WHY IT WAS FIRST

Every other question uses "did the screen change?" as its measuring stick. If that ruler is bent, everything measured with it is wrong. So it had to be checked before anything else.

THE ANSWER

Already run. The things that looked like ticking clocks turned out to be **progress bars** — they move when you make progress, not on every move. So the ruler is not bent. This question is finished and needs no owner.

Should it click on objects it can see, instead of random spots on the screen?

TODAY

The screen is 64 by 64. That is **4,096 possible places to click**. Our agent picks one at random. It is like trying to use a phone by tapping the glass with your eyes shut.

THE IDEA

Look at the screen first. Find the **shapes** — groups of touching pixels that share a colour. A game usually has a handful of them: a player piece, a door, a key, a wall. Click the middle of those shapes instead.

That turns 4,096 guesses into maybe 10 sensible ones.

HOW YOU TEST IT

CONTROL

Clicks a random spot out of all 4,096.

TREATMENT

Finds the shapes, clicks the centre of one of them.

Everything else stays identical: same games, same number of attempts, same action budget.

SUCCESS LOOKS LIKE

Clears clearly more levels than the control on the 17 held-out games — *and* the gain is spread over several games, not coming from one lucky one.

GIVE UP IF

On the 8 practice games it is no better than the control. Stop there and write it up.

WHAT COULD GO WRONG

- The background is one huge blob of colour, so your "shape finder" returns one useless giant shape.
- The thing you need to click is an empty space, not an object.
- The shapes move after every action, so your list is out of date the moment you make it.

Can it learn, during one game, which buttons actually do something?

TODAY

There are a few buttons. In any particular game, some of them do nothing at all. Our agent spreads its presses evenly across all of them and keeps wasting moves on the dead ones right until the budget runs out.

THE IDEA

Keep a small scoreboard while you play: for each button, how often did it change the screen or clear a level? Then spend more of your remaining moves on the buttons that are paying off, and fewer on the ones that are not.

This is a well-known recipe called a **bandit** — the name comes from choosing between slot machines when you do not know which one pays out.

HOW YOU TEST IT

CONTROL

Picks evenly among the buttons, the whole way through.

TREATMENT

Keeps the scoreboard and shifts towards the buttons that work.

SUCCESS LOOKS LIKE

More levels cleared than even picking, on the held-out games, spread across games.

GIVE UP IF

It matches or loses to plain even picking on the practice games.

WHAT COULD GO WRONG

- **It decides too early.** One lucky press makes it commit to a button and it never tries the others again.
- **There is not enough time to learn.** An attempt is only 300 actions. Learning which button works might eat most of that, leaving nothing to use the knowledge with.
- The right button changes between levels, so what it learned on level 1 is wrong on level 2.

If it remembers where it has been and walks back to unexplored places, does it get further?

TODAY

Random play wanders. It reaches an interesting screen, wanders away, and has no idea it was ever there. It goes over the same ground again and again and never systematically finishes exploring anywhere.

THE IDEA

Keep a list of screens you have already seen, and which moves you have already tried on each one. When you are stuck, deliberately travel back to a screen where untried moves remain, and try one of those.

In plain words: instead of wandering, explore like someone mapping a building — finish one room before moving on, and come back to the doors you did not open.

HOW YOU TEST IT

CONTROL

The same agent with no memory of where it has been.

TREATMENT

Remembers screens and returns to unfinished ones.

SUCCESS LOOKS LIKE

More levels cleared on the held-out games. A useful second measurement: how many *different* screens it reached. That tells you whether the exploring part worked even if the score did not move.

GIVE UP IF

It sees no more distinct screens than the control on the practice games. If it is not exploring more, the idea has already failed and the score does not matter.

WHAT COULD GO WRONG

- **You cannot always go back.** There is no undo button. Returning to a screen may be impossible, or may cost more actions than the trip is worth.
- Walking back eats the budget. You have 300 actions; spending 80 of them retracing your steps is expensive.
- Two different situations can look like the same screen, so your map quietly merges places that are not the same.

If it already knew the goal, how much better would it do?

THE IDEA

No code at all. You sit down with the practice games, play or watch them as a human, and work out what winning actually requires.

Then you count: **if a player already knew the goal, what is the smallest number of moves that would win?**

That number is the **ceiling** — the best any agent could possibly do on that game.

WHY THIS MATTERS MORE THAN IT SOUNDS

It tells us which half of the problem is hard.

- If someone who *knows* the goal still needs 200 moves, then guessing the goal better will not save us. The hard part is control, and that is where effort should go.
- If someone who knows the goal needs 12 moves, while random play needs thousands and usually fails, then the entire problem is *working out what the goal is* — and everything else is a distraction.

Nobody can answer that by running more code. Somebody has to look.

WHAT YOU PRODUCE

A table: one row per practice game, saying what the goal turned out to be, the shortest winning move count you found, and how confident you are. Plus a short note on what the games have in common.

WHO THIS SUITS

Anyone blocked on setup, or anyone who would rather think than debug installs. No GitHub, no Python, no Kaggle run, no GPU. It is the cheapest of the five and possibly the most informative.

WHAT COULD GO WRONG

- You guess a goal wrongly and your ceiling number is fiction. Say how sure you are, game by game.
- Only use the **8 practice games**. Looking at the 17 held-out games, even once, to "just check", ruins them for everyone.

All five, side by side

ID	IN ONE LINE	NEEDS	STATUS
P1	Is our "did the screen change?" test being fooled by ticking counters?	—	Closed. They were progress bars.
P2	Click the shapes you can see, instead of anywhere on the screen.	Laptop	Open
P3	Learn during the game which buttons pay off, and use them more.	Laptop	Open
P4	Remember where you have been and go back to unfinished places.	Laptop	Open
P5	Work out by hand how good a player who knew the goal could be.	Pen and paper	Open

BEFORE YOU RUN ANYTHING

The two sentences that make this real research

Whichever question you take, the preregistration form asks twelve things. Two of them are the whole method, and both are answered **before** a single run:

- **What number counts as success?**
- **What result makes me give up?**

Decide these after seeing the numbers and you will always find something that looks like success. Writing them first is the reason anyone will believe your answer.

A negative result is a real result

If your give-up rule fires, stop and write it up with the same care as a win. Three of the first five experiments ended exactly that way, and they are the most trustworthy part of the whole record.

Always report every game, not just the total

A doubled score once turned out to come almost entirely from one level of one game. The total said "strong method". The per-game table said "one game". Only one of those was true. Ask yourself directly: is my whole result coming from one lucky game?

Practice games can kill an idea. They can never prove one. Settings are chosen on the 8 practice games; the verdict comes only from the 17 held-out games, run once, exactly as written down.